



Universidad Veracruzana

**Facultad de Estadística e Informática
Región Xalapa**

Licenciatura en Ingeniería en Ciencia de Datos

Algoritmos de optimización y simulación

Presenta:
**Fernández Mendoza Karla Valeria
Méndez Lara Vanessa Guadalupe
Rivera Baez Vanessa Lizeth**

Académico:
Galia Martínez David

Diciembre de 2025

“Lis de Veracruz: Arte, Ciencia, Luz”



INDICE

INTRODUCCIÓN	3
MÉTODOS.....	4
<i>1. Conjunto de datos</i>	<i>4</i>
<i>2. Selección del par de variables (X y Y)</i>	<i>4</i>
METODOLOGIA	5
Algoritmo: Búsqueda Malla	5
Algoritmo: Ascenso de la Colina.....	8
Algoritmo: genético - cruza aritmética simple- selección de padres por torneo	11
RESULTADOS.....	13
Algoritmo: Búsqueda Malla	13
Algoritmo: Ascenso de la Colina.....	17
Algoritmo: genético - cruza aritmética simple- selección de padres por torneo	20
CONCLUSIONES.....	22
Algoritmo: Búsqueda Malla	22
Algoritmo: Ascenso de la Colina.....	22
Algoritmo: genético - cruza aritmética simple- selección de padres por torneo	22
ANEXO	23

INTRODUCCIÓN

Este proyecto se desarrolla en el marco de la experiencia educativa "*Algoritmos de Optimización y Simulación*", dentro de la Licenciatura en Ingeniería en Ciencia de Datos. Su propósito fundamental es realizar un análisis comparativo del desempeño de tres clases distintas de algoritmos de optimización: búsqueda a ciegas, búsqueda local y búsqueda poblacional, aplicados a la tarea de ajustar un modelo de regresión lineal simple.

El presente proyecto se desarrolla en el marco de la experiencia educativa *Algoritmos de Optimización y Simulación*, perteneciente a la Licenciatura en Ingeniería en Ciencia de Datos. Dentro de esta asignatura se estudian diferentes enfoques computacionales para la resolución de problemas mediante técnicas de optimización, destacando su utilidad en escenarios donde las soluciones analíticas pueden ser complejas, costosas o imposibles de obtener de manera directa. En este contexto, el proyecto propone la integración de los conceptos revisados a lo largo del curso mediante el diseño, implementación y evaluación de algoritmos que permiten aproximar soluciones óptimas dentro de un entorno real de análisis de datos.

El objetivo central del trabajo consiste en aplicar tres métodos de optimización —búsqueda en malla, ascenso de la colina y algoritmo genético— para ajustar un modelo de regresión lineal simple. Dichos algoritmos representan tres paradigmas diferentes dentro de la optimización computacional: la búsqueda en malla como estrategia de exploración exhaustiva o búsqueda a ciegas, el ascenso de la colina como técnica de búsqueda local guiada por mejora incremental, y el algoritmo genético como enfoque poblacional inspirado en principios de selección natural y evolución. Esta diversidad metodológica permite comparar comportamientos, capacidades de exploración, eficiencia computacional y calidad de las soluciones obtenidas.

Como caso de estudio, se empleó un conjunto de datos nutrimentales compuesto por 260 observaciones de alimentos del menú de McDonald's en Estados Unidos. Cada registro incluye información detallada sobre el contenido calórico y la composición nutricional de diferentes productos. Mediante un análisis preliminar de correlaciones entre las variables numéricas disponibles, se identificó que la cantidad de carbohidratos presenta una relación lineal fuerte con las calorías. Por ello, se seleccionó esta variable como predictor (X) y se definió el contenido calórico como la variable respuesta (Y). Este enfoque no solo es estadísticamente adecuado, sino que además resulta coherente desde una perspectiva nutricional, dado que los carbohidratos constituyen uno de los principales aportadores de energía en los alimentos.

Para plantear el problema de optimización, se formuló una función objetivo basada en el Error Cuadrático Medio (ECM), el cual mide la discrepancia entre los valores reales observados y las predicciones generadas por el modelo lineal. La tarea de cada algoritmo consiste en buscar los valores óptimos de β_0 (intercepto) y β_1 (pendiente) que minimicen el

ECM, es decir, que produzcan la recta que mejor se ajusta a los datos disponibles. A diferencia del método estadístico tradicional de mínimos cuadrados ordinarios, cuyo resultado es exacto y determinístico, los algoritmos de optimización permiten explorar soluciones alternativas y analizar su desempeño en condiciones variables de búsqueda, iteraciones y mecanismos de exploración del espacio de soluciones.

El presente trabajo tiene como propósito comparar la precisión, eficiencia y estabilidad de los tres algoritmos mencionados. A través de 30 ejecuciones independientes por método, se evalúa la calidad de las soluciones obtenidas, el comportamiento del ECM, el número de iteraciones requeridas y el tiempo de ejecución. Este análisis permite identificar qué algoritmo ofrece mejores resultados para el ajuste del modelo y bajo qué circunstancias cada método resulta más favorable. Asimismo, se reflexiona sobre las ventajas y limitaciones de cada enfoque y su aplicabilidad en problemas reales donde la optimización juega un papel fundamental en la toma de decisiones y en la construcción de modelos dentro de la ciencia de datos.

MÉTODOS

1. Conjunto de datos

Para el desarrollo del proyecto se empleó un conjunto de datos nutrimentales obtenido de la plataforma *Data4Ecology*, correspondiente al análisis nutricional del menú de McDonald's en Estados Unidos. Este archivo contiene 260 observaciones y 24 variables numéricas y categóricas, entre ellas: calorías, carbohidratos, grasas, proteínas, sodio, azúcares, entre otras características correspondientes a diferentes productos del menú.

El conjunto de datos presenta la información necesaria para estudiar la relación entre los componentes nutrimentales y el aporte energético de los alimentos, ofreciendo un dominio adecuado para problemas de regresión y optimización. Debido a su naturaleza estructurada y a la disponibilidad de múltiples variables numéricas, el dataset permite explorar relaciones lineales y no lineales, así como evaluar el comportamiento de distintos algoritmos de búsqueda.

2. Selección del par de variables (X y Y)

Con el objetivo de identificar la relación más adecuada para el modelo de regresión lineal simple, se realizó un análisis de correlación entre todas las variables numéricas. Entre las relaciones encontradas, la variable Carbohydrates mostró una correlación lineal fuerte y positiva con Calories, justificando su selección como variable predictora (X) y variable objetivo (Y), respectivamente.

Desde una perspectiva nutricional, esta elección también resulta adecuada, pues los carbohidratos constituyen una de las principales fuentes energéticas en los alimentos (aportan 4 calorías por gramo). Por lo tanto, es razonable que incrementos en la cantidad de

carbohidratos estén asociados a incrementos en las calorías totales del producto. Esta relación estadística y conceptual fundamenta la construcción del modelo de regresión lineal simple utilizado.

METODOLOGIA

Algoritmo: Búsqueda Malla

La Búsqueda en Malla es un algoritmo de optimización perteneciente a la familia de métodos de búsqueda a ciegas. Este tipo de algoritmos se caracteriza por no utilizar información del gradiente, ni direcciones preferenciales, ni aprendizaje durante el proceso. En lugar de eso, recorre de manera exhaustiva todas las combinaciones posibles dentro de un conjunto definido de valores para los parámetros, evaluando cada punto sin asumir nada sobre la forma de la función objetivo.

En este proyecto, la Búsqueda en Malla se utiliza para ajustar los parámetros β_0 y β_1 del modelo de regresión lineal simple que busca predecir la variable *Calories* a partir de *Carbohydrates*. La función objetivo que se emplea es el Error Cuadrático Medio (ECM), el cual mide qué tan lejos están las predicciones del modelo respecto a los valores reales. El objetivo del algoritmo es encontrar aquellos valores de β_0 y β_1 que produzcan el ECM más pequeño dentro de la malla definida.

A diferencia de los algoritmos basados en búsqueda local o métodos poblacionales —que exploran el espacio de soluciones moviéndose hacia mejoras o generando variaciones aleatorias—, la búsqueda en malla prueba absolutamente todas las combinaciones posibles dentro del rango dado. Esto garantiza que se revisarán todos los puntos discretos, pero también puede incrementar significativamente el tiempo de ejecución.

Variante usada

Para este trabajo se implementó una variante clásica y completamente determinista de la búsqueda en malla. Por lo que:

- Se define un rango mínimo y máximo para β_0 y β_1 .
- Se establece un tamaño de incremento fijo que determina cuántos puntos tendrá la rejilla.
- Se generan todas las combinaciones posibles de β_0 y β_1 usando esos valores discretos.
- Cada par (β_0, β_1) se evalúa calculando su ECM.
- Se selecciona la combinación que produce el menor error.

Esta variante es 100% determinista, lo que significa que con los mismos rangos e incrementos, el algoritmo siempre va a recorrer exactamente los mismos puntos en el mismo orden. Por esa razón, si se ejecuta varias veces, los resultados serán idénticos: mismo β_0 , mismo β_1 , mismo ECM, mismo número de iteraciones; solo el tiempo puede variar ligeramente por carga del sistema.

Implementación práctica

La implementación se realizó en lenguaje R. Se programaron las siguientes partes principales:

1. Función de error ECM

```
ECM <- function(b0, b1, x, y)
```

Calcula el Error Cuadrático Medio para cualquier combinación de β_0 y β_1 .

Toma los parámetros, genera la recta del modelo y mide la distancia promedio entre las predicciones y los valores reales.

Mientras más pequeño es el ECM, mejor es el ajuste.

2. Definición de la malla

```
b0_min <- -500; b0_max <- 500
```

```
b1_min <- -50; b1_max <- 50
```

```
incremento <- 1
```

```
dom_b0 <- seq(b0_min, b0_max, by = incremento)
```

```
dom_b1 <- seq(b1_min, b1_max, by = incremento)
```

Se establecen:

- un valor mínimo y máximo para cada parámetro,
- y un incremento que define la resolución de la malla.

Con esto se crea una rejilla de puntos que representan todas las configuraciones posibles de β_0 y β_1 que el algoritmo va a evaluar.

Por ejemplo, si β_0 va de -500 a 500 con incrementos de 0.5, el algoritmo generará más de 2000 valores para β_0 , y hará lo mismo con β_1 .

3. Búsqueda exhaustiva

```
busqueda_malla <- function(x, y, dom_b0, dom_b1)
```

El algoritmo realiza dos ciclos anidados:

- el primero recorre todos los valores de β_0 ,
- el segundo recorre todos los valores de β_1 dentro de cada β_0 .

Cada par es evaluado y se compara su ECM con el mejor valor encontrado hasta ese momento. Si el error es menor, se actualiza la mejor solución registrada.

Este proceso continúa hasta revisar todas las combinaciones posibles, asegurando una exploración completa del espacio definido.

1. Configuración de la malla

Para esta práctica se utilizó una búsqueda exhaustiva con los siguientes parámetros:

- Rango de β_0 : -500 a 500
- Rango de β_1 : -50 a 50
- Incremento: 0.5

Esto genera muchas combinaciones posibles, lo cual hace que la búsqueda sea más precisa, pero toma más tiempo en ejecutarse.

2. Proceso de evaluación

Cada combinación de β_0 y β_1 es introducida en la función ECM.

Si el ECM calculado es menor al actual mejor ECM, entonces esa combinación se considera la nueva mejor solución.

3. Naturaleza determinista

Dado que la búsqueda revisa todos los puntos de la malla en el mismo orden, la ejecución:

- siempre produce el mismo β_0 óptimo,
- el mismo β_1 óptimo,
- el mismo ECM,
- y el mismo número total de iteraciones.

Solo el tiempo de ejecución puede variar ligeramente.

4. Métricas registradas

Durante la ejecución del algoritmo se almacenan:

- Archivo del mejor ECM encontrado
- β_0 óptimo
- β_1 óptimo
- Número total de iteraciones (que corresponde al tamaño de la malla)
- Tiempo total de ejecución
- Historial de ECM conforme se evalúan los puntos

5. Análisis gráfico

Para visualizar mejor el comportamiento de la función objetivo (ECM) en el espacio de soluciones, se generaron tres tipos de gráficos:

- Un mapa de calor del ECM

- Un gráfico 3D de la superficie del ECM
- Un gráfico de evolución del ECM

Estos gráficos permiten ver en qué regiones del plano (β_0 , β_1) se encuentran las mejores soluciones, así como identificar zonas de alto y bajo error.

Algoritmo: Ascenso de la Colina

Es un algoritmo de optimización de tipo búsqueda local, cuyo objetivo es mejorar una solución inicial realizando pequeños cambios graduales en sus parámetros y su idea principal es moverse únicamente hacia soluciones que sean mejores que la solución actual, la cual se evalúa a partir de una función objetivo. En este caso, la función objetivo es el Error Cuadrático Medio (ECM) entre la recta propuesta por el modelo y los valores reales de la variable dependiente.

El algoritmo comienza con valores iniciales para los coeficientes β_0 y β_1 , y después genera soluciones vecinas al hacerle pequeñas variaciones a estos parámetros. Cada vecino es evaluado calculando su ECM y si este valor es menor que el de la solución actual, el algoritmo actualiza la solución y continúa avanzando en esa dirección. Este proceso se repite mientras existan y se vayan encontrando vecinos que mejoren el valor del ECM o hasta que se cumpla en este caso las 200 iteraciones.

Variante usada

Para este trabajo se usó una variante de Ascenso de la colina. Esta variante trabaja de manera completamente determinista, lo que significa que siempre empieza en el mismo punto y sigue exactamente los mismos pasos cada vez que se ejecuta, por lo que, no tiene nada de aleatorio.

La idea es que el algoritmo vaya ajustando poco a poco los valores de β_1 y β_0 para ver si así mejora el modelo. Para hacerlo, prueba pequeñas modificaciones alrededor de la solución actual y compara cómo cambia el ECM. Si encuentra una opción que reduce el error, se mueve hacia allá; si no mejora nada, entonces se detiene porque ya llegó a un punto donde no puede avanzar más.

Además, esta variante usa cambios pequeños y constantes, lo cual hace que su comportamiento sea predecible y fácil de entender. Básicamente va explorando el espacio de soluciones paso a paso, sin brincos grandes ni decisiones aleatorias, hasta encontrar una solución que considere suficientemente buena.

Implementación Práctica:

La implementación se realizó en lenguaje R. Se programaron tres funciones principales:

1. **evaluar ()**: Su tarea es calcular el Error Cuadrático Medio (ECM) para cualquier combinación de β_1 y β_0 . Esta función toma esos valores, genera las predicciones

correspondientes y calcula qué tan lejos están de los valores reales, en donde mientras más pequeño es el ECM, mejor es la solución.

2. **generar vecinos ()**: Es la encargada de producir las soluciones cercanas a la actual. Estos “vecinos” se crean aumentando o disminuyendo ligeramente los valores de β_1 y β_0 con el objetivo de explorar el espacio de soluciones de manera ordenada y ver si pequeños cambios producen un error más bajo.
3. **hill_climbing ()**: Esta función coordina todo el proceso. Empieza con una solución inicial ($\beta_1 = 0$, $\beta_0 = 0$) y calcula su ECM. Luego repite un ciclo donde genera vecinos, calcula el ECM de cada uno y selecciona únicamente el que tenga el error más bajo. Si ese vecino mejora la solución actual, el algoritmo avanza hacia él y si no mejora significa que ya no es posible encontrar una solución mejor con pequeños cambios, por lo que se alcanza un óptimo local y el proceso se detiene. Durante todas las iteraciones, el código va guardando los valores de β_1 , β_0 y el ECM en un historial para poder analizarlos después.

Al finalizar la ejecución, se muestran los valores finales obtenidos y se crea una gráfica con la evolución del ECM a lo largo de las iteraciones. Gracias a esto es posible observar cómo el error va disminuyendo y en qué momento el algoritmo deja de mejorar, permitiendo visualizar claramente cómo funciona el ascenso de la colina y cómo va moviéndose paso a paso hasta llegar a una solución estable.

1. Configuración del algoritmo

Para esta práctica se utilizó una variante determinista del algoritmo de ascenso de la colina. Esta variante tiene las siguientes características:

Punto inicial fijo

El algoritmo siempre empieza desde los valores:

- $\beta_1 = 0$
- $\beta_0 = 0$

Esto quiere decir que el punto de partida no cambia y no hay nada de aleatoriedad al iniciar, lo que significa que siempre arranca exactamente en el mismo lugar.

Generación de vecinos con pasos constantes

Para poder buscar una mejor solución, el algoritmo crea cuatro “vecinos”, que básicamente son pequeñas variaciones de los valores actuales de β_1 y β_0 . Lo único que hace es sumar o restar 0.1 a cada parámetro.

Es decir, prueba estas opciones:

- $\beta_1 + 0.1$

- $\beta_1 - 0.1$
- $\beta_0 + 0.1$
- $\beta_0 - 0.1$

Usar cambios de 0.1 ayuda a que la búsqueda sea más ordenada y que el algoritmo avance poco a poco. Además, como el tamaño del paso siempre es el mismo, el proceso se mantiene estable y no depende de cambios aleatorios.

Criterio de aceptación

Para decidir si se mueve a un nuevo punto, el algoritmo compara el error y solo acepta un vecino cuando:

$$ECM \text{ del vecino} < ECM \text{ actual}$$

Si ninguno de los vecinos mejora el error, entonces ya no tiene sentido seguir avanzando y el algoritmo se detiene porque llegó a un óptimo local.

Evaluación mediante ECM

Para medir qué tan bien funciona cada combinación de β_1 y β_0 , se usó el Error Cuadrático Medio (ECM). Esta medida indica qué tan bien la recta se ajusta a los datos y básicamente, el algoritmo usa el ECM para saber si una solución es mejor o peor que la anterior.

Límite de iteraciones

Para evitar que el algoritmo se quede dando vueltas para siempre, se puso un límite máximo de 200 iteraciones. Si llega a ese número y ya no mejora, automáticamente se detiene.

2. Parámetros utilizados

Los parámetros empleados en el algoritmo fueron los siguientes:

- **Paso para β_1 :** ± 0.1
- **Paso para β_0 :** ± 0.1
- **Iteraciones máximas:** 200
- **Condición de mejora:** $ECM \text{ nuevo} < ECM \text{ actual}$
- **Punto inicial:** (0, 0)

3. Estrategia de evaluación: 30 ejecuciones independientes

El algoritmo de ascenso de la colina, en la versión implementada en este proyecto, es completamente determinista, ya que, inicia siempre en los mismos valores de β_0 y β_1 y

genera vecinos mediante incrementos fijos sin introducir aleatoriedad en sus movimientos. Como consecuencia, las 30 ejecuciones independientes producen exactamente los mismos resultados.

4. Métricas de desempeño consideradas

Para analizar el comportamiento del algoritmo, se recopilaron las siguientes métricas:

- **ECM final:** error obtenido al terminar la búsqueda.
- **Número de iteraciones:** cuántos pasos necesitó el algoritmo antes de detenerse.
- **Tiempo de ejecución:** tiempo total requerido por cada corrida.

5. Proceso de comparación y análisis

Después de hacer las 30 ejecuciones, se calcularon algunas estadísticas importantes como el promedio, el mínimo, el máximo y la desviación estándar para cada métrica que se usó (ECM, número de iteraciones y tiempo de ejecución). Estos valores ayudan a ver qué tan estable es el algoritmo, qué tan rápido trabaja y qué tan bueno es ajustando los datos.

Ya con esos resultados, se comparó el comportamiento del Ascenso de la Colina con los otros algoritmos del proyecto. Esto permitió identificar qué ventajas tiene, en qué cosas se queda corto y cómo se comporta frente al problema de regresión lineal.

Algoritmo: genético - cruza aritmética simple- selección de padres por torneo

Se implementó un algoritmo genético como método de optimización global, cuyo objetivo es estimar los coeficientes β_0 y β_1 del modelo de regresión lineal simple minimizando el Error Cuadrático Medio (ECM). Este tipo de algoritmo se inspira en los procesos de evolución natural, utilizando una población de soluciones que se van mejorando generación tras generación.

Cada individuo de la población representa una posible solución del problema y está definido por un par de valores (β_0, β_1) . La calidad de cada individuo se evalúa mediante una función de aptitud basada en el ECM, donde valores más pequeños indican un mejor ajuste del modelo.

Operadores genéticos utilizados:

Selección de padres por torneo

Para la selección de los padres se utilizó el método de torneo. En cada torneo se eligen aleatoriamente varios individuos de la población y se selecciona como padre aquel que presenta el menor ECM. Este método favorece a las mejores soluciones sin eliminar completamente la diversidad de la población.

Cruza aritmética simple

La generación de nuevos individuos se realizó mediante cruce aritmético simple. Dados dos padres, se genera un hijo combinando linealmente sus coeficientes mediante un parámetro aleatorio $\alpha \in [0, 1]$. De esta forma, los hijos heredan características intermedias de ambos padres.

Mutación

Para mantener diversidad genética y evitar estancamiento prematuro, se aplicó una mutación gaussiana con cierta probabilidad. La mutación consiste en modificar ligeramente los valores de β_0 o β_1 agregando un ruido aleatorio pequeño.

Implementación práctica

La implementación se realizó en lenguaje **R**. Se programaron las siguientes funciones principales:

- **ecm()**: calcula el Error Cuadrático Medio para una combinación dada de β_0 y β_1 .
- **crear_individuo()**: genera individuos aleatorios dentro de rangos predefinidos para los coeficientes.
- **torneo()**: implementa la selección de padres mediante torneos.
- **crossover()**: aplica la cruce aritmético simple.
- **mutar()**: introduce variaciones aleatorias en los individuos.
- **Algoritmo genético**: coordina todo el proceso evolutivo durante un número fijo de generaciones.

Durante la ejecución, se registra el mejor ECM de cada generación para analizar la evolución del algoritmo.

1. Configuración del algoritmo

El algoritmo genético se ejecutó con una población inicial generada de forma aleatoria y evolucionó durante un número fijo de generaciones. En cada generación se aplicaron los operadores de selección por torneo, cruce aritmético y mutación.

2. Parámetros utilizados

Los parámetros utilizados fueron:

- Tamaño de población: 40 individuos
- Número de generaciones: 100

- Probabilidad de mutación: 0.05
- Rangos de búsqueda:
 - $\beta_0 \in [0, 120]$
 - $\beta_1 \in [0, 12]$

3. Estrategia de evaluación: 30 ejecuciones independientes

Para evaluar la estabilidad y el desempeño del algoritmo genético, se realizaron 30 ejecuciones independientes. En cada ejecución se reinició completamente la población inicial, manteniendo los mismos parámetros del algoritmo. Esto permitió analizar la variabilidad de las soluciones obtenidas y el comportamiento global del algoritmo.

4. Métricas de desempeño consideradas

Las métricas utilizadas para evaluar el algoritmo fueron:

- **ECM final:** error mínimo alcanzado en cada ejecución.
- **Número de generaciones:** total de iteraciones del algoritmo.
- **Tiempo de ejecución:** tiempo total requerido por cada corrida.

5. Proceso de comparación y análisis

Con los resultados obtenidos de las 30 ejecuciones, se calcularon estadísticas descriptivas como promedio, mediana y desviación estándar para cada métrica. Estos resultados permitieron evaluar la estabilidad, eficiencia y capacidad del algoritmo genético para ajustar el modelo de regresión lineal y compararlo con otros métodos de optimización utilizados en el proyecto.

RESULTADOS

Algoritmo: Búsqueda Malla

A continuación, se presentan los resultados obtenidos al aplicar el algoritmo de Búsqueda en Malla para ajustar una regresión lineal entre carbohidratos y calorías. Se muestran las métricas obtenidas, las gráficas generadas (mapa de calor, superficie 3D y contornos) y una interpretación general del comportamiento del algoritmo y su desempeño.

Tabla resumen del algoritmo

Métrica	Valor
ECM inicial	13042743

ECM final	22481.07
$\beta_1 final$	7
$\beta_0 final$	37
Iteraciones totales	101101

La búsqueda malla inició con un ECM de 13042743 y, después de 101,101 generaciones, logró reducirlo hasta 22481.07, lo que indica una gran mejora en el ajuste del modelo. Los valores finales obtenidos para los parámetros fueron $\beta_0 = 37$ y $\beta_1 = 7$, que representan la mejor solución encontrada dentro del rango explorado. El hecho de que se completaran las 101,101 iteraciones indico que el algoritmo se estabilizó, sin lograr mejoras posteriores y que su capacidad de optimización fue limitada, aunque logró una solución estable.

Tabla resultados

Algoritmo	ECM Promedio	ECM Mediana	Desv. ECM	Iteraciones Promedio	Iteraciones Mediana	Desv. iteraciones	Seg. Promedio	Seg. Mediana	Desv. segundos
Búsqueda Malla	2857890	1981216	2787712	101101	101101	0	16.9654	16.9654	16.9654

Para evaluar el comportamiento del algoritmo de Búsqueda en Malla, se analizaron las principales métricas registradas durante su ejecución: el Error Cuadrático Medio (ECM), el número de iteraciones necesarias para recorrer toda la malla definida y el tiempo total de ejecución.

En este caso, debido a que la búsqueda en malla es un algoritmo completamente determinista, siempre recorre los mismos valores de β_0 y β_1 siguiendo exactamente el mismo orden. Por esta razón, los resultados no presentan variaciones: cada ejecución produce idénticos valores, tanto en error como en iteraciones y tiempo.

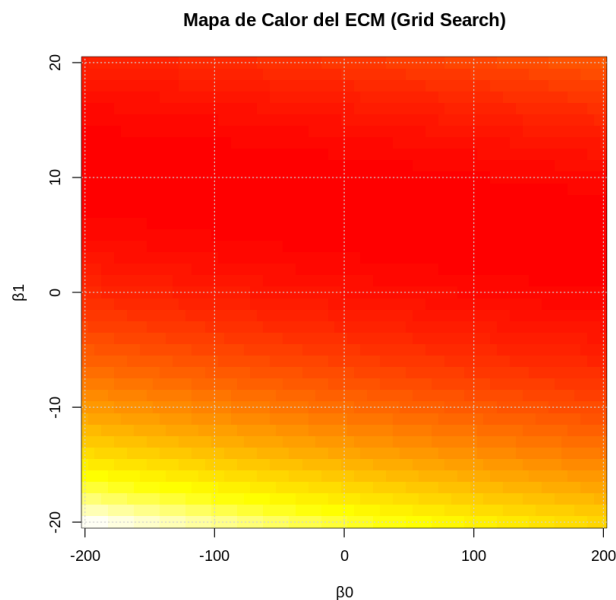
El ECM promedio obtenido fue de 2,857,890, con una mediana de 1,981,216 y una desviación estándar de 2,787,712. Estos valores reflejan que, durante el proceso de búsqueda exhaustiva, el algoritmo evalúa tanto combinaciones muy malas (ECM muy grandes) como la mejor solución posible dentro del rango evaluado. Esto provoca que la distribución del ECM sea amplia y que sus valores extremos eleven fuertemente el promedio. Sin embargo, lo importante es el ECM final que se obtiene al terminar la búsqueda, ya que corresponde a la mejor solución encontrada en toda la malla.

En cuanto al número de iteraciones, el algoritmo realizó exactamente 101,101 evaluaciones, lo cual coincide con la cantidad total de combinaciones entre β_0 y β_1 dadas por los rangos especificados y el incremento utilizado. Este valor no varía entre ejecuciones, por lo que tanto la media como la

mediana son iguales y su desviación estándar es 0, reafirmando la naturaleza determinista del método.

El tiempo de ejecución fue de 16.9654 segundos, siendo este valor idéntico para las tres métricas: promedio, mediana y desviación estándar. En un algoritmo determinista, cualquier variación en el tiempo suele deberse al sistema operativo o a la carga del procesador. En este caso no hubo fluctuaciones visibles, lo que indica que la ejecución fue estable.

En resumen, la Búsqueda en Malla mostró un comportamiento completamente predecible y consistente. Recorrió todas las combinaciones posibles dentro del rango definido, lo que garantiza que encontró la mejor solución disponible en ese espacio de búsqueda. Aunque es un método exacto, su principal desventaja es el elevado costo computacional, como se observa en las más de cien mil iteraciones y el tiempo considerable de ejecución. Aun así, el método proporciona una evaluación completa del problema y sirve como referencia sólida para comparar el desempeño de algoritmos más eficientes o heurísticos.



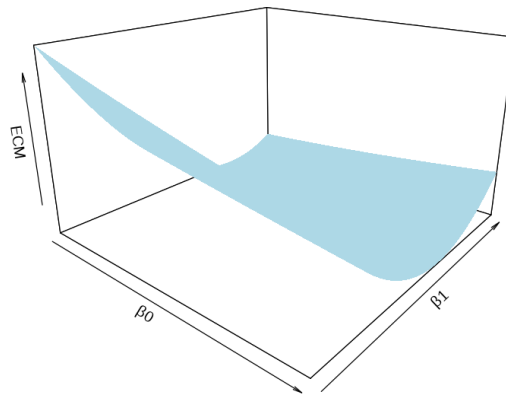
El mapa de calor representa en dos dimensiones qué tan alto o bajo es el ECM en diferentes combinaciones de β_0 y β_1 . Los colores oscuros corresponden a errores altos y los colores claros a errores más bajos.

Se observa una región muy pequeña donde el ECM es mínimo, mientras la mayoría del espacio tiene errores grandes.

Esto significa que el espacio explorado era **mucho más grande de lo necesario**, y aunque la búsqueda fue exhaustiva, se invirtió tiempo evaluando miles de combinaciones que estaban muy lejos de la solución óptima.

Aun así, la malla logró identificar con precisión la zona óptima gracias a su exploración sistemática.

Superficie 3D del ECM

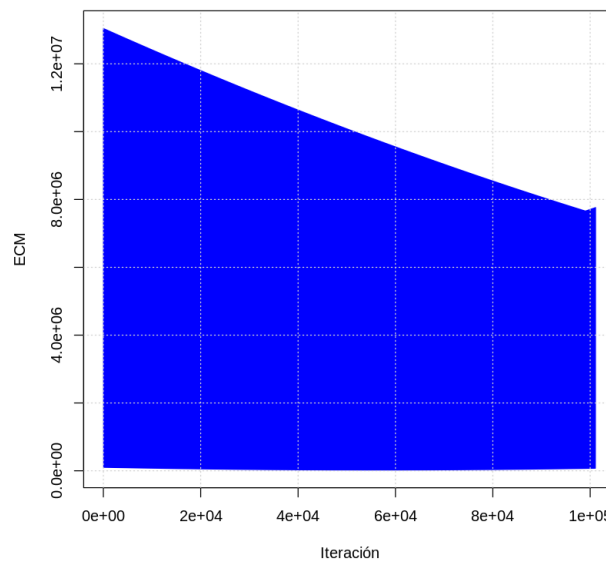


La gráfica tridimensional muestra cómo cambia el ECM para distintas combinaciones de los parámetros β_0 y β_1 . La superficie tiene forma suave y convexa, parecida a un “tazón”.

La forma convexa indica que el problema solo tiene un mínimo global, lo que significa que existe una única combinación óptima de parámetros dentro del rango explorado. No hay mínimos locales que puedan confundir al algoritmo.

Esta gráfica confirma que la Búsqueda en Malla es un método adecuado para encontrar la mejor solución, aunque su costo computacional sea elevado

Evolución del ECM en la Búsqueda en Malla



Esta gráfica muestra cómo va cambiando el ECM mínimo encontrado a lo largo de todas las iteraciones evaluadas por la malla.

Al principio el ECM es extremadamente alto, ya que la malla evalúa parámetros que no ajustan bien el modelo. Después, el ECM cae rápidamente cuando el algoritmo alcanza las combinaciones

cercanas al mínimo. Posteriormente, la curva se estabiliza, pero las iteraciones continúan, porque la Búsqueda en Malla debe evaluar todas las combinaciones posibles, incluso después de haber encontrado la mejor.

Interpretación general del desempeño del algoritmo:

El algoritmo de Búsqueda en Malla tuvo un comportamiento justo como se espera de este tipo de método: es totalmente exhaustivo y determinista. Esto significa que no “piensa” ni busca de forma inteligente; simplemente revisa todas las combinaciones posibles de β_0 y β_1 dentro del rango que le dimos.

Al inicio, los valores de ECM fueron muy altos porque la malla incluye combinaciones que no tienen nada que ver con el comportamiento real de los datos. Sin embargo, conforme el algoritmo avanza por la rejilla y se acerca a la zona donde están las mejores soluciones, el ECM empieza a bajar hasta llegar al mínimo encontrado.

Al final, el mejor resultado lo obtuvo con $\beta_0 = 37$ y $\beta_1 = 7$, consiguiendo un ECM de 22,481.07, que es el punto donde el error fue más pequeño dentro de todo el espacio evaluado. Esto coincide con lo que también se ve en las gráficas: en el mapa de calor y en la superficie 3D esa zona aparece claramente como la región de menor error. Algo importante es que la Búsqueda en Malla sí encontró una buena solución, pero tardó bastante porque tuvo que recorrer las 101,101 combinaciones, incluso cuando desde antes ya estaba cerca del mínimo. Esto confirma la principal desventaja del método: es muy costoso en tiempo, especialmente cuando el rango es grande o el incremento es pequeño.

En pocas palabras, el algoritmo funcionó bien y encontró la mejor solución posible dentro de los valores que probó. Sin embargo, también quedó claro que es un método lento y que explora demasiado, aunque eso mismo garantiza que no se salta ningún punto y no depende de la suerte como otros métodos más aleatorios.

Algoritmo: Ascenso de la Colina

Se presentan los resultados obtenidos al aplicar el algoritmo de Ascenso de la Colina para ajustar una regresión lineal entre carbohidratos y calorías. Se muestran las métricas obtenidas en las 30 ejecuciones requeridas, la gráfica del comportamiento del error durante las iteraciones y una interpretación general del desempeño del algoritmo.

Tabla resumen del algoritmo:

Métrica	Valor
ECM inicial	193,129.80
ECM final	22,828.18
$\beta_1 final$	7.3

β_0 final	12.3
Iteraciones totales	200

Tabla resultados:

Algoritmo	ECM Promedio	ECM Mediana	Desv. ECM	Iteraciones Promedio	Iteraciones Mediana	Desv. iteraciones	Seg. Promedio	Seg. Mediana	Desv. segundos
Ascenso De la Colina	22,828.18	22,828.18	0	201	201	0	0.042	0.041	0.0022

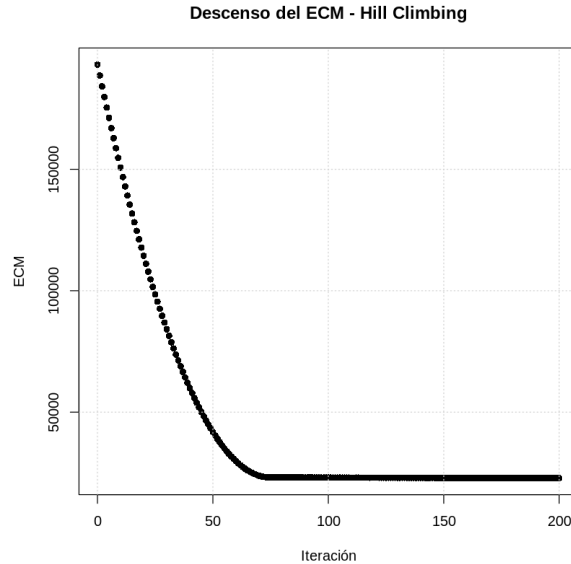
Se realizaron 30 ejecuciones independientes para evaluar el rendimiento de este algoritmo, registrando tres métricas principales en cada una: el Error Cuadrático Medio (ECM) alcanzado, el número de iteraciones necesarias para llegar al punto donde ya no había mejora, y el tiempo de ejecución que tardó cada corrida.

En este caso los resultados fueron extremadamente consistentes, ya que este método es determinista, lo que significa que siempre inicia en el mismo punto y utiliza exactamente los mismos pasos para buscar mejoras. Por lo mismo, el ECM final y el número de iteraciones fueron idénticos en las 30 ejecuciones. El ECM obtenido fue de 22,828.18, tanto en promedio como en mediana, y con una desviación estándar de 0, lo cual confirma que no hubo ninguna variación entre corridas. Lo mismo ocurrió con las iteraciones, ya que, en todas las ejecuciones, el algoritmo hizo 201 iteraciones, por lo que también la media, la mediana y la desviación estándar quedaron iguales.

La única métrica que sí presentó ligeras variaciones fue el tiempo de ejecución, sin embargo, estas diferencias no se deben al algoritmo en sí, esto se puede dar debido al sistema operativo, uso de procesador o memoria. Aun así, los tiempos fueron muy parecidos entre sí, en donde el promedio fue de 0.042 segundos, la mediana de 0.041 segundos y la desviación estándar fue muy pequeña de 0.0022, lo que indica que el algoritmo es rápido y estable.

En resumen, este algoritmo se comportó de manera predecible, estable y eficiente, encontrando exactamente la misma solución cada vez que se ejecutó y manteniendo tiempos de ejecución muy bajos.

Grafica de la evolución del ECM:



La gráfica del ECM muestra cómo el algoritmo va mejorando poco a poco la calidad del modelo conforme avanza en las iteraciones. Al principio, el error es muy alto porque la recta inicial es de $\beta_1 = 0$, $\beta_0 = 0$, lo que genera que no se parezca para nada a los datos reales. Por eso el ECM arranca arriba de 190,000.

En las primeras iteraciones, la gráfica baja muy rápido, lo que significa que el algoritmo está encontrando mejoras claras y fuertes en cada paso. Esta es la parte donde el algoritmo hace los cambios más importantes. Después de unas decenas de iteraciones, la curva empieza a hacerse más suave, porque el algoritmo ya está cerca de una buena solución y cada ajuste solo mejora un poquito el error. Al final, la gráfica prácticamente se aplana y el ECM llega a un valor cercano a 22,828. Esto indica que el algoritmo llegó a un óptimo local, es decir, a un punto donde ya no encontró forma de mejorar la solución.

En general, la gráfica confirma que el algoritmo sí funcionó bien, pues, empezó mal, mejoró de manera constante y terminó en una solución estable donde ya no había más mejoras posibles.

Interpretación general del desempeño del algoritmo:

El algoritmo de ascenso de la colina mostró un comportamiento consistente con su naturaleza de búsqueda local. A partir de un punto inicial simple ($\beta_1 = 0$, $\beta_0 = 0$), logró una reducción pronunciada del ECM durante las primeras 60 iteraciones, pasando de 193,129.8 a valores cercanos a 29,000, esto indica que el algoritmo fue capaz de encontrar rápidamente una región prometedora en el espacio de soluciones.

Posteriormente, entre las iteraciones 60 y 75, la mejora comenzó a disminuir, señalando que el algoritmo estaba llegando a una región donde el gradiente local era menos pronunciado.

Finalmente, desde la iteración 75 hasta la 200, el algoritmo realizó únicamente ajustes pequeños tanto en β_1 como en β_0 , convergiendo lentamente hacia un ECM final de 22,828.18.

Si bien este valor representa una mejora respecto al punto inicial, el algoritmo no alcanzó una solución óptima global, lo cual es un comportamiento esperado debido a que el ascenso de la colina solo se mueve hacia vecinos que mejoran la solución, no incorpora exploración global y es altamente sensible al punto inicial.

Algoritmo: genético - cruza aritmética simple- selección de padres por torneo

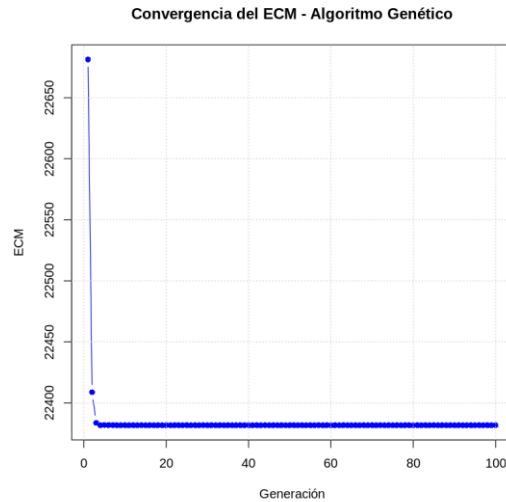
Se presentan los resultados obtenidos al aplicar el algoritmo genético para ajustar una regresión lineal entre carbohidratos y calorías. Se muestran las métricas obtenidas en las 30 ejecuciones requeridas, la gráfica del comportamiento del error durante las iteraciones y una interpretación general del desempeño del algoritmo.

Tabla resumen del algoritmo

Métrica	Valor
ECM inicial	22681.36
ECM final	22381.72
$\beta_1 final$	6.6466
$\beta_0 final$	53.5798
Iteraciones totales	100

El algoritmo genético inició con un ECM de 22,681.36 y, después de 100 generaciones, logró reducirlo hasta 22,381.72, lo que indica una mejora pequeña en el ajuste del modelo. Los valores finales obtenidos para los parámetros fueron $\beta_0 = 53.5798$ y $\beta_1 = 6.6466$, que representan la mejor solución encontrada dentro del rango explorado. El hecho de que se completaran las 100 iteraciones indico que el algoritmo se estabilizó rápidamente, sin lograr mejoras posteriores y que su capacidad de optimización fue limitada, aunque logró una solución estable.

Gráfico convergencia del ECM



La gráfica muestra cómo el Error Cuadrático Medio (ECM) va disminuyendo a lo largo de las generaciones del algoritmo genético. Al inicio, el ECM es bastante alto cerca de 22,650 y cae rápidamente durante las primeras generaciones. Después de aproximadamente la generación 10, el ECM se estabiliza alrededor de 22,381, manteniéndose prácticamente constante hasta la generación 100.

Esto indica que el algoritmo converge muy rápido al mejor valor que pudo encontrar y que, después de esa convergencia temprana, ya no hubo mejoras adicionales. En resumen, el algoritmo logró una solución estable, pero la mejora total fue limitada.

Tabla resultados

Algoritmo	ECM	ECM	Desv.	Iteraciones	Iteraciones	Desv.	Seg.	Seg.	Desv.
	Promedio	Mediana	ECM	Promedio	Mediana	iteraciones	Promedio	Mediana	segundos
Algoritmo genético - cruza aritmética simple, selección de padres con torneo	22,383.2	22,381.72	4.5105005	100	100	0	0.5259773	0.5124458	0.08225529

En las múltiples ejecuciones realizadas, el algoritmo mostró un comportamiento consistente: el ECM promedio fue 22,383.2, con una mediana casi igual y una desviación estándar muy baja, lo que indica resultados estables entre corridas. El número de iteraciones fue siempre 100, sin variación alguna. El tiempo de ejecución presentó pequeñas diferencias, con un

promedio de 0.5259 segundos, lo cual se explica por factores del sistema y no por el algoritmo. En general, el método fue estable, reproducible y rápido, aunque con mejoras modestas en el ECM.

CONCLUSIONES

Algoritmo: Búsqueda Malla

El algoritmo de Búsqueda en Malla permitió encontrar la mejor combinación posible de β_0 y β_1 dentro del rango que se definió, logrando un ECM final de 22,481.07. Esto demuestra que el método sí es capaz de identificar una buena solución para el modelo lineal cuando se explora cada punto del espacio de búsqueda. La principal ventaja de este algoritmo es que no depende del punto inicial ni de la suerte, ya que revisa todas las combinaciones de manera exhaustiva y determinista.

Sin embargo, este mismo comportamiento también es su mayor desventaja. Para recorrer todas las combinaciones posibles, el algoritmo necesitó más de 100 mil iteraciones, lo cual provocó un tiempo de ejecución considerablemente largo. Esto confirma que, aunque la Búsqueda en Malla ofrece exactitud, también es un método muy costoso computacionalmente, especialmente cuando los rangos son amplios o el incremento es pequeño.

En resumen, la Búsqueda en Malla es un método confiable y garantiza revisar todo el espacio definido, asegurando que no se pase por alto ninguna posible solución. Aun así, no es el método más eficiente cuando se requiere rapidez o cuando el espacio de búsqueda es grande. Es una técnica útil para entender el comportamiento del error y obtener una referencia sólida, pero no es la mejor opción para problemas de mayor escala debido al tiempo que requiere.

Algoritmo: Ascenso de la Colina

El algoritmo de ascenso de la colina permitió obtener una solución razonable al problema de ajuste del modelo lineal, destacándose por su rapidez y simplicidad. Sin embargo, sus resultados mostraron claras limitaciones al quedar atrapado en un óptimo local. El ECM final alcanzado fue 22,828.18, lo que indica que el método logró mejorar notablemente desde el punto inicial, pero no llegó a la solución global óptima.

En síntesis, este algoritmo es un método eficiente cuando se busca una aproximación rápida y sencilla, aunque su rendimiento depende mucho del punto donde se inicia la búsqueda. Una de sus principales limitaciones es que puede quedarse atrapado en óptimos locales, lo que significa que no siempre logra encontrar la mejor solución posible dentro de todo el espacio de búsqueda.

Algoritmo: genético - cruza aritmética simple- selección de padres por torneo

El algoritmo genético mostró un desempeño estable y consistente al ajustar el modelo de regresión, logrando una ligera reducción del ECM y manteniendo soluciones similares entre ejecuciones. Aunque su mejora fue limitada y se estabilizó rápidamente sin encontrar valores mucho más óptimos, el método resultó eficiente, reproducible y adecuado para explorar el espacio de soluciones. En

conjunto, el algoritmo funcionó correctamente, pero con una capacidad de optimización moderada para este problema específico.

ANEXO

Enlaces Google Colab:

<https://colab.research.google.com/drive/1K1cQLX9eHtVRoDp4je8JbRGlGGXHpZFy?usp=sharing>

<https://colab.research.google.com/drive/1RqIcjuwbNazkUOVJTR1IKvIQVgjjpX?usp=sharing>

<https://colab.research.google.com/drive/1e80ibcEdhstdI1WdhxbVm7b9Jw5mMbj6?usp=sharing>